

Date e orari

Date e orari

Lavorare con date e orari usando il modulo datetime in Python.

Perché è utile usare date e orari in modo corretto

Qualunque programma reale prima o poi ha a che fare con le date: "quando è stata inviata questa email?", "quanti giorni mancano al compleanno?", "quanti giorni fa ho creato questo file?". Python ha un modulo dedicato per tutto questo.

Il modulo `datetime`

Il modulo `datetime` offre quattro strumenti principali:

Classe	Cosa rappresenta
<code>date</code>	Solo la data (anno, mese, giorno)
<code>time</code>	Solo l'orario (ore, minuti, secondi)
<code>datetime</code>	Data e orario insieme
<code>timedelta</code>	Una durata — la differenza tra due momenti

```
from datetime import date, datetime, timedelta # Importa solo quello che serve
```

La data e l'ora di adesso

```
from datetime import date, datetime

oggi = date.today()      # Solo la data odierna
print(oggi)              # 2024-01-15 (formato: anno-mese-giorno)

adesso = datetime.now()  # Data e orario attuali
print(adesso)           # 2024-01-15 14:30:25.123456
```

Creare una data specifica

```
from datetime import date, datetime

compleanno = date(2008, 3, 25)      # anno=2008, mese=3, giorno=25
print(compleanno)                  # 2008-03-25

capodanno = datetime(2025, 1, 1, 0, 0, 0) # anno, mese, giorno, ora,
                                         # minuti, secondi
print(capodanno)                  # 2025-01-01 00:00:00
```

Accedere alle singole parti di una data

```
from datetime import datetime

adesso = datetime.now()

print(adesso.year)      # Anno (es: 2024)
print(adesso.month)     # Mese (1-12)
print(adesso.day)       # Giorno (1-31)
print(adesso.hour)      # Ora (0-23)
print(adesso.minute)    # Minuti (0-59)
print(adesso.second)    # Secondi (0-59)
```

Formattare una data come testo: `strftime()`

Per default le date si stampano nel formato `2024-01-15`. Con `strftime()` puoi scegliere tu come vuoi che appaia:

```
from datetime import datetime

adesso = datetime.now()

print(adesso.strftime("%d/%m/%Y"))      # 15/01/2024 (formato italiano)
print(adesso.strftime("%d %B %Y"))      # 15 January 2024 (con nome del
mese)
print(adesso.strftime("%H:%M:%S"))      # 14:30:25 (solo l'orario)
print(adesso.strftime("%d/%m/%Y %H:%M")) # 15/01/2024 14:30
```

I codici di formato principali:

Codice	Significato	Esempio
%Y	Anno (4 cifre)	2024
%m	Mese (2 cifre)	01
%d	Giorno (2 cifre)	15
%H	Ora in formato 24h	14
%M	Minuti	30
%S	Secondi	25
%A	Nome del giorno (inglese)	Monday
%B	Nome del mese (inglese)	January

Convertire testo in data: `strptime()`

L'operazione inversa: prendi una stringa di testo e trasformala in un oggetto data. Devi dirle in che formato è scritta la stringa:

```
from datetime import datetime

testo = "25/03/2008"
data = datetime.strptime(testo, "%d/%m/%Y") # Dici a Python come è
formattata la stringa
print(data)          # 2008-03-25 00:00:00
print(data.year)     # 2008
```

Calcolare differenze tra date: `timedelta`

`timedelta` rappresenta una **durata** — un certo numero di giorni, ore, minuti, ecc. Puoi aggiungerla o sottrarla a una data:

```

from datetime import date, timedelta

oggi = date.today()

# Tra quanti giorni sarà la prossima settimana?
tra_una_settimana = oggi + timedelta(days=7)
print(tra_una_settimana)

# Che giorno era 30 giorni fa?
trenta_giorni_fa = oggi - timedelta(days=30)
print(trenta_giorni_fa)

# Quanti giorni sono passati dal tuo compleanno?
nato = date(2008, 3, 25)
differenza = date.today() - nato      # Sottrai due date → ottieni un
timedelta
print(f"Giorni vissuti: {differenza.days}")

```

Confrontare date

Le date si confrontano come i numeri — con `<`, `>`, `==` :

```

from datetime import date

d1 = date(2024, 1, 1)
d2 = date(2024, 12, 31)

print(d1 < d2)    # True ← d1 viene prima di d2
print(d1 == d2)   # False ← sono date diverse

if date.today() > d1:
    print("Siamo dopo il 1 gennaio 2024")

```